ML LAB PROJECT REPORT

PhishGuard

Machine Learning Based Phishing URL Detection

Group Members

Sr. No.	Name	Roll No.
1	Parnil Kulkarni	A4-B1-14
2	Ayush Pediwal	A4-B1-16
3	Shreyash Mishra	A4-B2-18
4	Arshpreet Puri	A4-B2-24

1. Introduction

Phishing is a common cybersecurity threat in which attackers create deceptive websites or URLs to trick users into revealing sensitive information such as passwords, banking details, and personal data. Since URLs often contain structural patterns that differ from legitimate websites, machine learning can be used to identify suspicious URLs automatically.

PhishGuard is a machine learning-based phishing URL detection system that classifies a given URL as either **Phishing** or **Legitimate**. The system uses lexical and structural characteristics of URLs such as URL length, number of dots, HTTPS usage, entropy, number of digits, subdomain count, query parameters, and domain-related properties.

Five machine learning algorithms were evaluated:

- Logistic Regression
- Decision Tree
- Random Forest
- Support Vector Machine (SVM)
- XGBoost

The best-performing model was then integrated into a Streamlit-based web application. The application allows a user to enter a URL and immediately receive a prediction along with the corresponding probability.

Objective: To develop a lightweight and accessible system for detecting potentially malicious URLs using machine learning.

2. Literature Review

Machine learning-based phishing detection has been widely studied because traditional rule-based systems may not be sufficient to detect continuously changing phishing URLs. The key studies reviewed for this project are summarised in Table 1.

Table 1: Summary of Literature Review

Research / Year	Focus	Key Contribution	Limitation / Research Gap
Reyes-Dorta et al. (2024)	Malicious URL Detection	Investigated malicious URL detection using machine learning and demonstrated the usefulness of URL-based characteristics for identifying malicious links, supporting lexical and structural URL features.	Centred on URL-feature usefulness; does not present a lightweight, user-facing deployment.
Phishing URL Detection Using ML and DL (2024)	ML vs Deep Learning Comparison	Compared traditional machine learning with deep learning; showed deep learning can perform strongly while feature-engineering-based ML also gives effective results.	Deep learning adds model complexity; lightweight engineered-feature approaches receive less emphasis.
Integrated DL + Transformer Model (2024)	Deep Learning with Pre-trained Transformer	Proposed an integrated model combining deep learning classifiers with a pre-trained transformer, showing the potential of advanced representation learning for URL classification.	Complex architecture; less suited to simple, resource-light systems.
Comparative ML Study (2025)	Classical ML Algorithm Comparison	Evaluated Logistic Regression, KNN, SVC, Naive Bayes, Decision Tree and Random Forest; Random Forest achieved strong results, showing ensembles suit phishing detection.	Limited to classical algorithms; boosting methods and web deployment are not covered.
Boakai et al. (2025)	ML Models with URL-Based Features	Compared machine learning models for phishing website detection using URL-based features; Random Forest and SVM performed strongly.	Primarily a model comparison; end-to-end application integration is not the focus.

Recent research therefore indicates that URL-based features combined with machine learning provide an effective approach for phishing detection, while more advanced deep learning and transformer-based methods can be considered for future improvements.

Research Gap

Many existing studies focus either on complex deep learning architectures or on specific datasets. PhishGuard focuses on developing a comparatively lightweight system using engineered URL features and multiple machine learning classifiers, followed by deployment through a simple web interface.

3. Methodology

The overall workflow of PhishGuard is:

> **Dataset → Data Cleaning → Exploratory Data Analysis → Train/Test Split → Model Training → Performance Evaluation → Best Model Selection → Feature Extraction → Streamlit UI → Deployment**

3.1 Dataset

The dataset used in the project contains 101,219 URL records with URL-related features and a target class.

Source: https://data.mendeley.com/datasets/hx4m73v2sf/2

The dataset contains:

- 1 URL column
- 16 URL-based features
- 1 target column (ClassLabel)

The features are described in Table 2.

Table 2: URL-Based Features Used

No.	Feature	Description
1	URL Length	Total length of the URL
2	Has IP Address	Indicates whether an IP address is used
3	Dot Count	Number of dots in the URL
4	HTTPS Flag	Indicates HTTPS usage
5	URL Entropy	Measures randomness of characters
6	Token Count	Number of URL tokens
7	Subdomain Count	Number of subdomains
8	Query Parameter Count	Number of query parameters
9	TLD Length	Length of the top-level domain
10	Path Length	Length of URL path
11	Hyphen in Domain	Indicates a hyphen in the domain
12	Number of Digits	Number of numerical characters
13	TLD Popularity	Indicates common TLD usage
14	Suspicious File Extension	Detects suspicious extensions
15	Domain Name Length	Length of domain
16	Percentage Numeric Characters	Percentage of numerical characters

4. Data Cleaning and Preprocessing

The dataset was inspected for missing values and duplicate records before model training. The following cleaning operations were performed:

1. Missing target values were removed.
2. Duplicate records were removed.
3. The target column was separated from the input features.
4. The dataset was divided into training and testing sets.

The train-test split used an 80:20 ratio with random_state = 42 and stratification to maintain the class distribution.

4.1 Dataset Statistics

The dataset initially contained:

- 101,219 records
- 1 missing target value
- 346 duplicate records

After cleaning, approximately 100,872 records remained.

4.2 Class Distribution

The original class distribution was:

- Class 0: 63,678
- Class 1: 37,540

During analysis, Class 0 was mapped to Phishing and Class 1 to Legitimate.

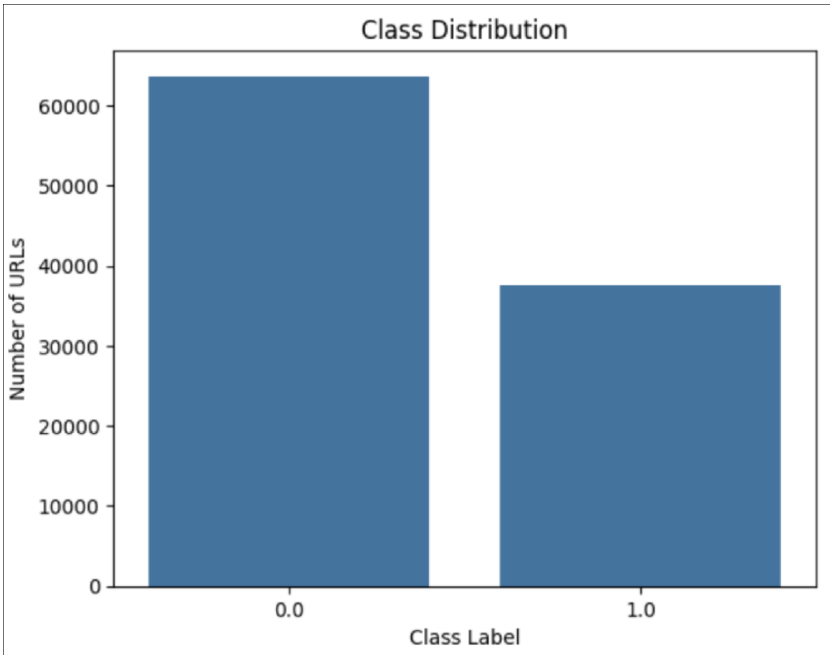


Figure 1: Class Distribution Graph

5. Exploratory Data Analysis

Exploratory data analysis was performed to understand the characteristics of phishing and legitimate URLs. A correlation heatmap was generated to observe relationships between numerical URL features.

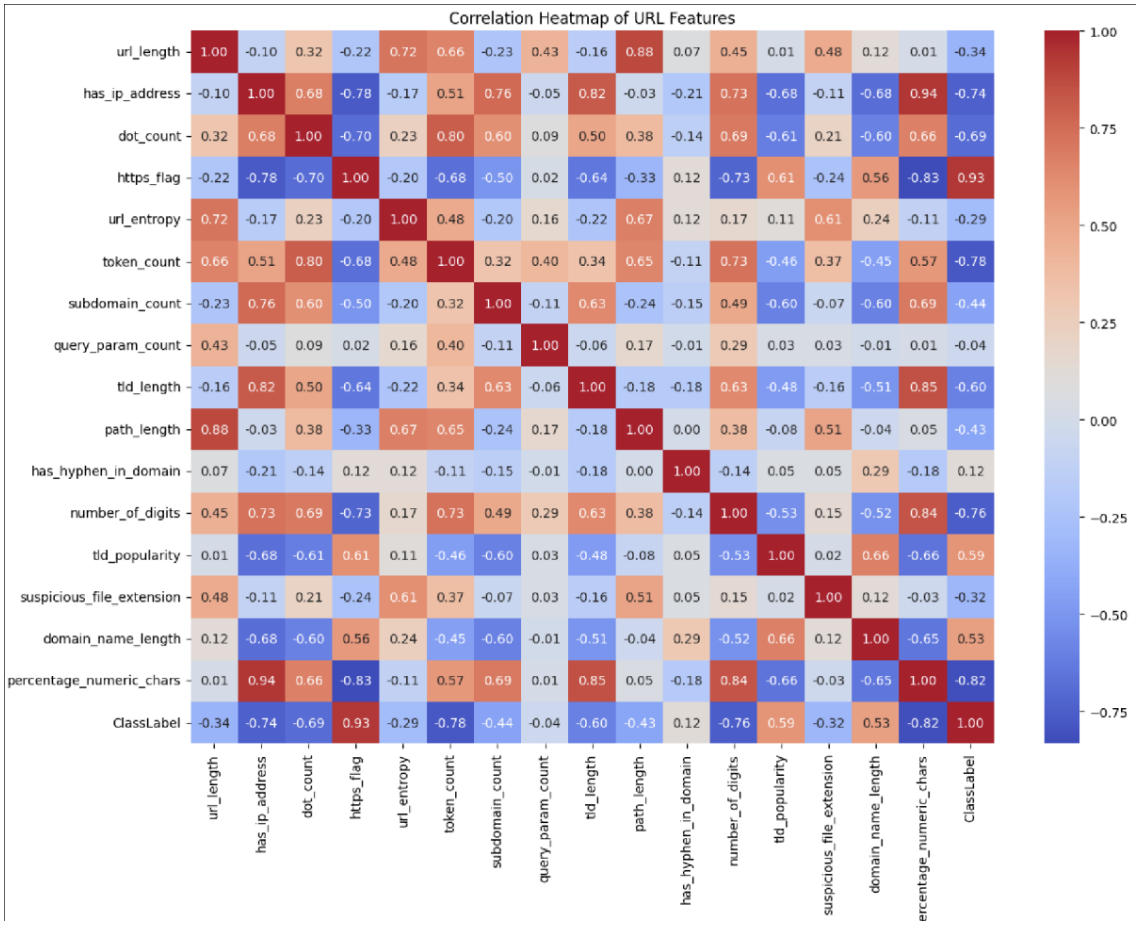


Figure 2: Correlation Heatmap

Feature distributions were also examined using histograms and boxplots. Features such as URL length, URL entropy, domain length, and number of digits showed differences between phishing and legitimate URLs.

For example, suspicious URLs may contain unusual lengths, higher numerical character usage, additional subdomains, or uncommon structural patterns.

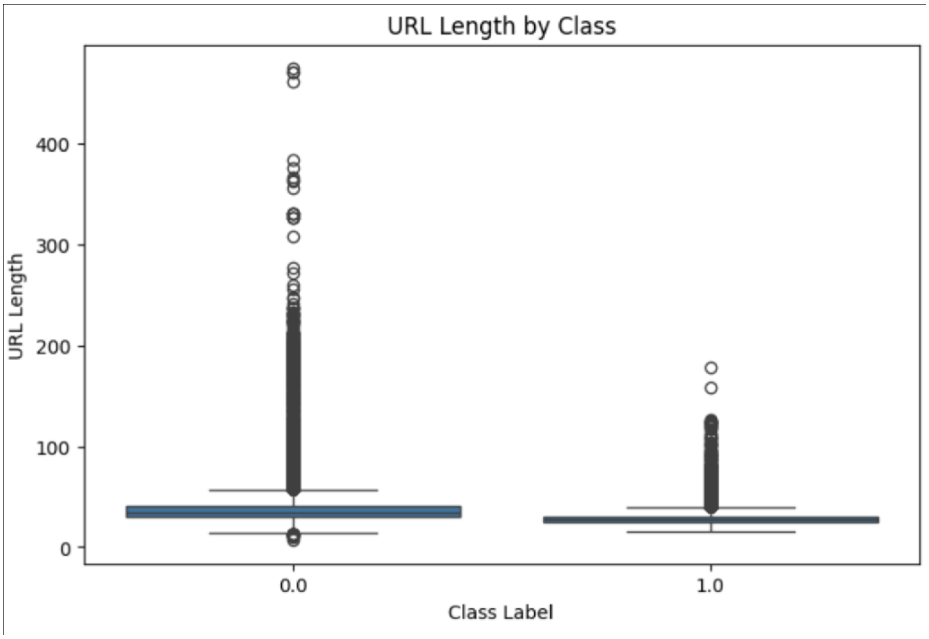


Figure 3: Feature Distribution / Boxplot

These observations supported the use of URL-based features for machine learning classification.

6. Machine Learning Models

Five classification algorithms were trained and evaluated.

6.1 Logistic Regression

Logistic Regression provides a simple baseline classification model. It estimates the probability of a URL belonging to a particular class.

6.2 Decision Tree

Decision Tree classifies data using a sequence of feature-based decisions. It is easy to interpret and can capture nonlinear relationships.

6.3 Random Forest

Random Forest combines multiple decision trees and uses their combined predictions to improve classification performance and reduce overfitting.

6.4 Support Vector Machine

SVM attempts to find an optimal decision boundary between classes. Feature scaling was applied before SVM training.

6.5 XGBoost

XGBoost is an ensemble boosting algorithm that builds multiple decision trees sequentially to improve prediction performance.

7. Results and Discussion

The models were evaluated using accuracy, precision, recall, F1-score and ROC-AUC. The obtained results are shown in Table 3.

Table 3: Model Performance Comparison

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Random Forest	**99.9554%**	**99.9866%**	**99.8930%**	**99.9398%**	**0.999997**
XGBoost	99.9405%	99.9465%	99.8930%	99.9197%	0.999997
Decision Tree	99.9405%	99.9465%	99.8930%	99.9197%	0.999307
SVM	99.8563%	99.6798%	99.9331%	99.8063%	0.999952
Logistic Regression	99.7026%	99.3873%	99.8127%	99.5996%	0.999895

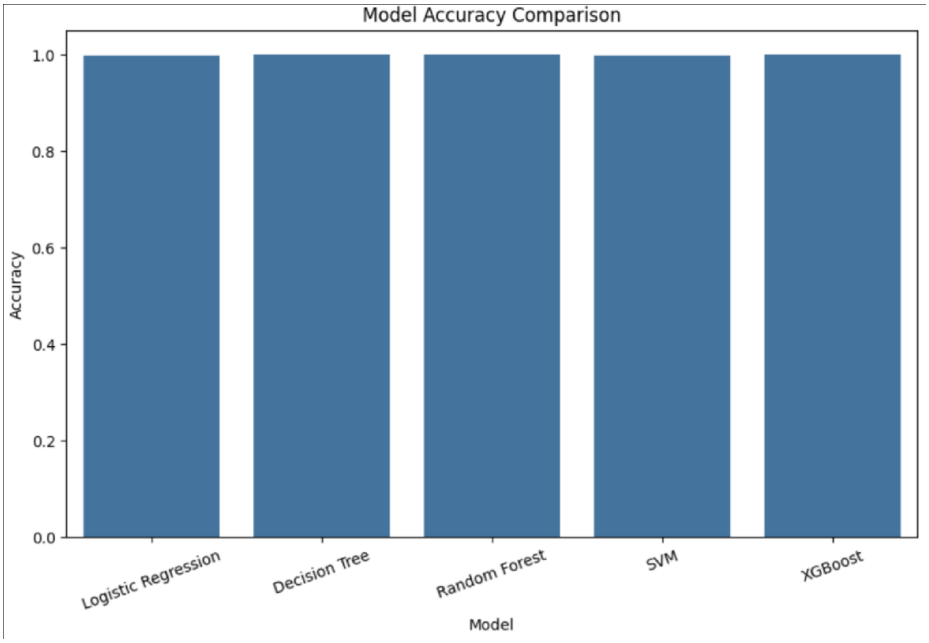


Figure 4: Model Performance Comparison Graph

Random Forest achieved the highest overall accuracy, precision, and F1-score. Its accuracy was 99.9554%, while its ROC-AUC was 0.999997.

SVM achieved the highest recall at 99.9331%, meaning it successfully identified a slightly higher proportion of positive samples. However, Random Forest provided the best overall balance of the evaluated metrics.

Therefore, Random Forest was selected as the final model for PhishGuard.

> *Note: These results represent the performance obtained on the project's dataset and test split and should not be interpreted as being universally superior to all existing state-of-the-art systems.*

8. Confusion Matrix and ROC Analysis

A confusion matrix was used to analyse the classification errors of the selected Random Forest model. It provides four outcomes:

- True Positive
- True Negative
- False Positive
- False Negative

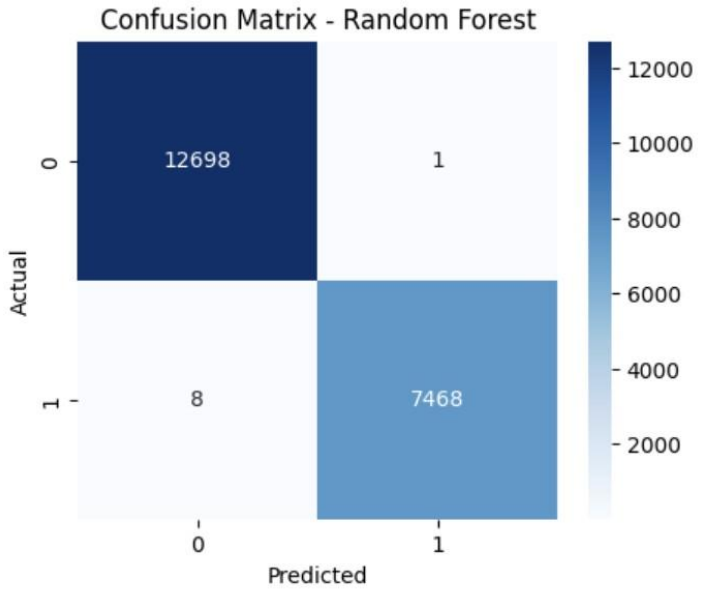


Figure 5: Random Forest Confusion Matrix

The very high accuracy and F1-score indicate that the model made very few classification errors on the test dataset.

ROC-AUC was also used to evaluate the model's ability to distinguish between the two classes. Random Forest achieved a ROC-AUC of 0.999997, indicating excellent separation between the classes.

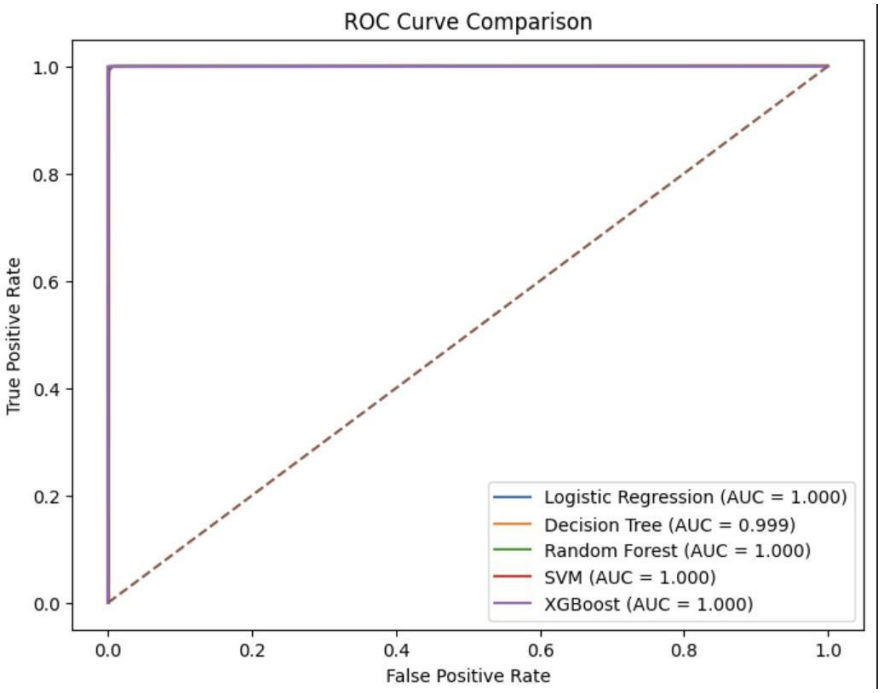


Figure 6: ROC Curve

9. User Interface Development

A web interface was developed using Streamlit. The interface allows the user to:

1. Enter a URL.
2. Extract the required URL features automatically.
3. Pass the features to the trained Random Forest model.
4. Display whether the URL is phishing or legitimate.
5. Display the corresponding prediction probability.

The application performs feature extraction directly from the entered URL using Python. The extracted features include URL length, HTTPS usage, domain characteristics, entropy, number of digits, subdomains, query parameters, and other URL properties.

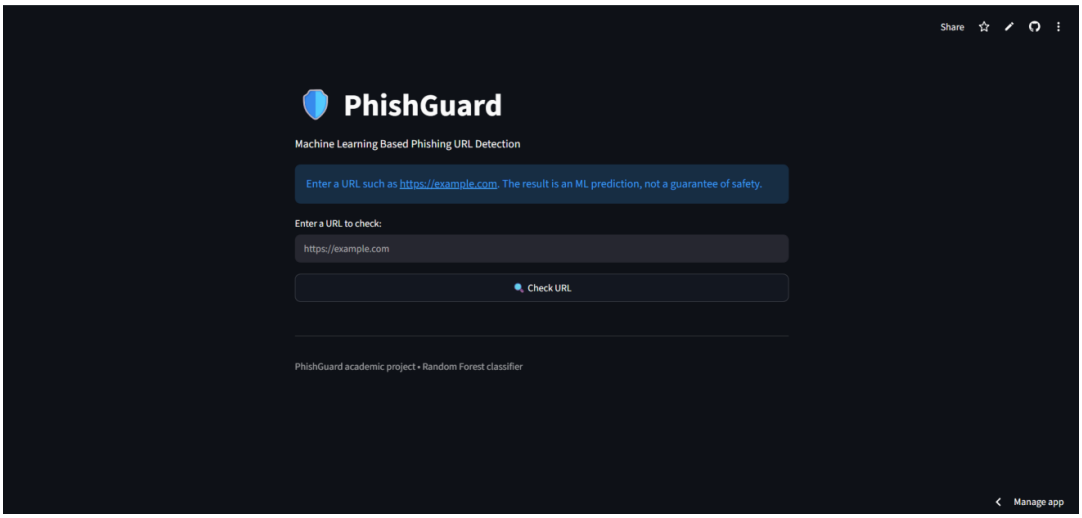


Figure 7: PhishGuard Streamlit Home Page

10. Prediction and Deployment

When a user enters a URL, PhishGuard processes it through the following sequence:

> **Input URL → Feature Extraction → Random Forest Model → Prediction → Probability Display**

For a phishing URL, the application displays a warning message. For a legitimate URL, it displays a successful classification message.

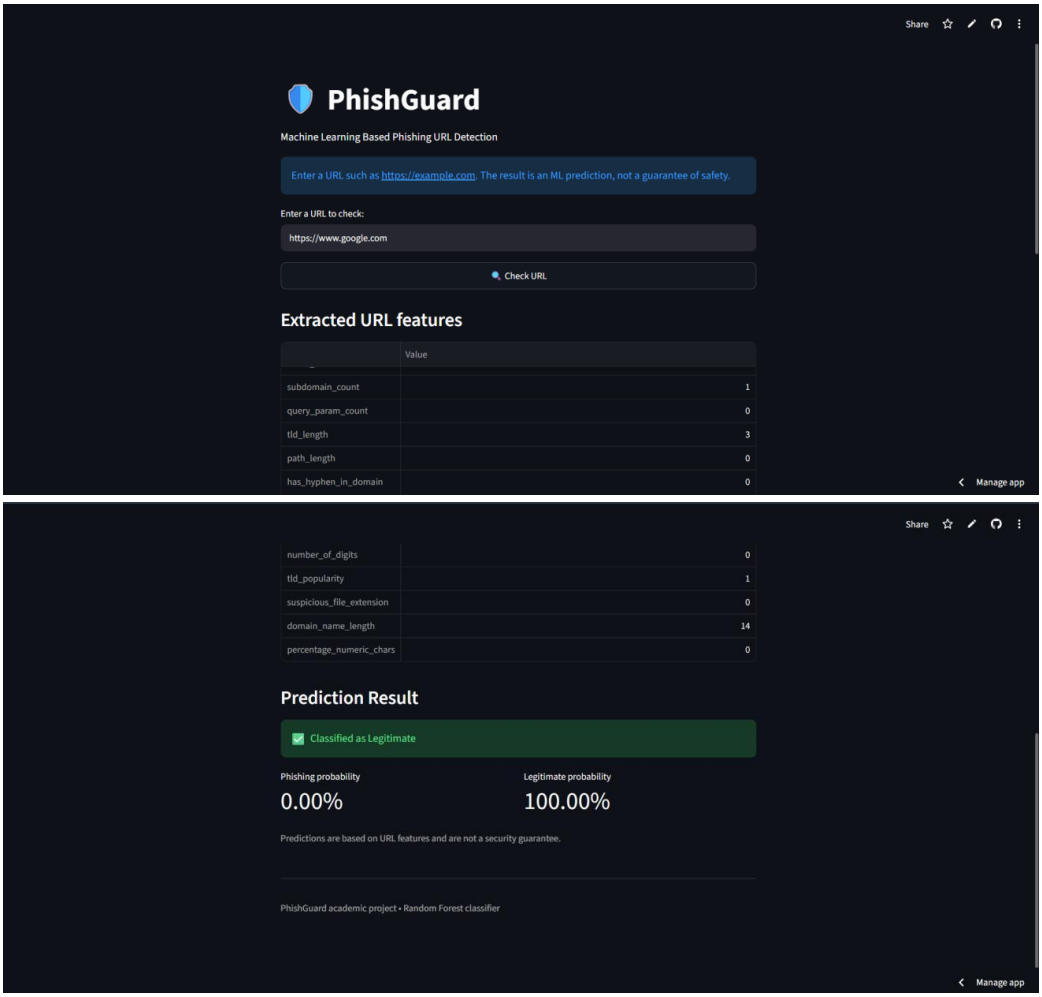


Figure 8: Legitimate URL Prediction

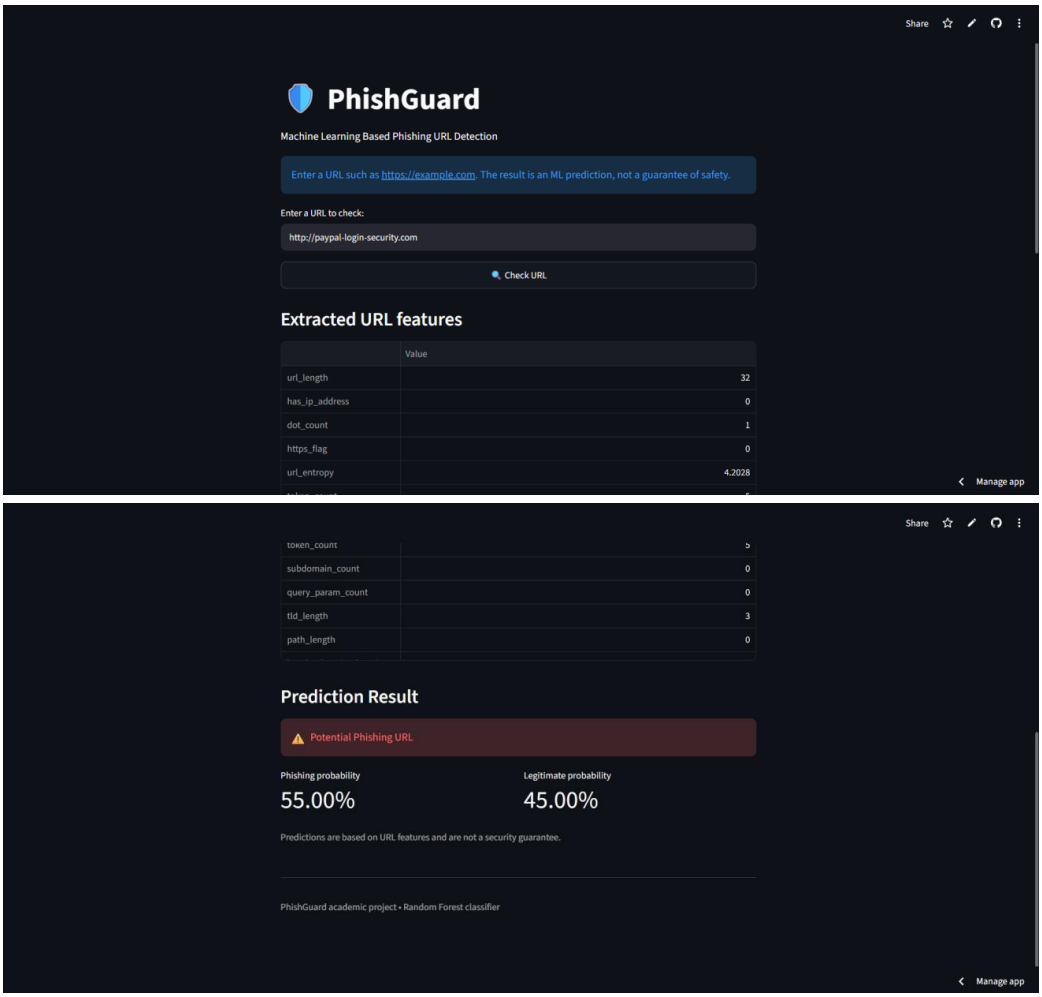


Figure 9: Phishing URL Prediction

The application was tested locally using Streamlit and subsequently deployed on Streamlit Community Cloud, making it accessible through a public URL.

Live application: https://phishingwebsitedetection-nihd5vfytcebr2dsbrzzr2.streamlit.app/

The deployment integrates the trained Random Forest model saved in .pkl format with the Streamlit web interface, enabling users to submit URLs and receive phishing or legitimate classifications. The application also displays the extracted URL features used for prediction. Unlike a Cloudflare Quick Tunnel, Streamlit Community Cloud hosts the application independently of the user's local machine, allowing public access without requiring the local Streamlit server to remain running.

The deployment demonstrates the integration of the trained machine learning model with a functional, publicly accessible web application rather than limiting predictions to a notebook-based environment.

11. Code Quality and GitHub Repository

The project was organized into a simple deployment-ready structure:

```
PhishGuard/
│
├── README.md
├── app.py
├── phishguard_model.pkl
└── requirements.txt
```

Main Components

File	Description
app.py	Contains the Streamlit application and URL feature extraction logic.
phishguard_model.pkl	Contains the trained Random Forest model.
requirements.txt	Contains the Python libraries required to run the project.
README.md	Contains project information, features, technologies, model performance, and instructions for running the application.

The project was uploaded to GitHub to provide version control and make the implementation easier to access and reproduce.

GitHub Link: https://github.com/Parnil-26/PhishingWebsiteDetection

12. Conclusion

PhishGuard demonstrates the use of machine learning for automated phishing URL detection. The project uses URL-based lexical and structural features and compares five machine learning algorithms.

Among the evaluated models, Random Forest performed the best overall, achieving 99.9554% accuracy, 99.9866% precision, 99.9398% F1-score, and 0.999997 ROC-AUC on the project test set. The selected model was integrated into a Streamlit web application that allows users to enter URLs and receive predictions in real time.

The project shows that engineered URL features combined with machine learning can provide an effective and lightweight approach to phishing URL classification.

Future Work

Future work could explore larger and more diverse datasets, additional URL and webpage features, deep learning models, real-time threat intelligence, and improved deployment infrastructure.

References

[1] N. Reyes-Dorta, P. Caballero-Gil, and C. Rosa-Remedios, “Detection of malicious URLs using machine learning,” Wireless Networks, vol. 30, 2024, pp. 7543–7560.

[2] “Phishing URL Detection Using Machine Learning and Deep Learning,” IEEE World AI IoT Congress, 2024.

[3] “An integrated model based on deep learning classifiers and pre-trained transformer for phishing URL detection,” Future Generation Computer Systems, vol. 161, 2024, pp. 269–285.

[4] “Phishing URL Detection Using Machine Learning: A Comparative Study,” 6th International Conference on Machine Learning and Computer Science Innovation, 2025.

[5] Boakai et al., “Comparative Evaluation of Machine Learning Models for Phishing Website Detection Using URL-Based Features,” IEEE Communications Standards Magazine, 2025.

[6] Islam et al., “Phishing URL Detection via Machine Learning: A Comprehensive Survey,” 2025.

[7] Kustiawan and Ghauth, “Evaluating the Impact of Feature Engineering in Phishing URL Detection: A Comparative Study of URL, HTML, and Derived Features,” IEEE Access, 2025.

[8] Mukunthan et al., “Comparative Analysis of Machine Learning Algorithms for Phishing Detection Using URL Features,” 2025.